SOURCE-GROUNDED REFACTORING STUDY

Shorter Lean Proofs for Diophantine Consistency

Concrete simplifications, reusable interfaces, checked prototypes, and a migration plan

Prepared for Vladimir Reshetnikov
26 October 2026

Reviewed repository. VladimirReshetnikov/ProveIt, `Computability/DiophantineConsistency`.
Pinned commit. `60e9c5d620c6c387d932876cfa2d9376f64ea56c`.
Toolchain. Lean 4.32.0; Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`.

Main recommendation. Start with existing library lemmas, common arithmetic identities, and the interfaces between proof stages. Then extract the common circuit tail and scanner simulation. These changes remove repeated reasoning while preserving the exact executable syntax, finite certificates, and theorem hypotheses.

The package contains reproducible source measurements, concrete Lean examples, patches, and a record distinguishing successful kernel checks from proposed integrations. It does not claim a completed refactoring of the whole repository or a new end-to-end consistency theorem.

This report reviews the active Lean tree, not every archived manuscript copy. All quantitative observations refer to the pinned snapshot.

Executive assessment

The best opportunities are concentrated in repeated mathematical interfaces: encoded-list scanners, coordinate renamings, finite coefficient rows, and the composition of already certified circuit blocks. A uniform substitution of `simp`, `aesop`, or a larger heartbeat limit would miss these targets.

There are useful changes at three scales. First, seven private list-lookup proofs can use two existing Lean-core lemmas; the supplied patch removes 40 physical lines. Second, a common arithmetic identity removes the half-radix case analysis, and one row-wise argument can replace hundreds of lines of repeated coefficient bounds. Third, the Split and Gap natural circuits have an exactly matching 99-node suffix under a specified register renaming. A reusable block for that suffix is the most concrete larger architectural target identified here.

Some abstractions are worthwhile without shortening the file immediately. For example, a common topological-DAG invariant theorem can replace repeated strong inductions. After charging for the new helper, the first few clients do not yet yield a net line saving. The report states this explicitly instead of treating every abstraction as a compression result.

Several elementary examples were checked with the pinned Lean compiler. Larger replacements and full scanner/circuit migrations have separate status labels in Section 17 and the archive's verification record. The strongest source comparisons are accompanied by scripts and exact data.

Contents

Executive assessment **1**

1 Scope, baseline, and what should be preserved **3**

2 Prioritized proposals **4**

3 Immediate reuse: eliminate seven private lookup proofs **4**

4 Move renaming out of expression evaluation **5**

5 Replace half-radix case analysis with one identity **6**

6 Compress the coefficient-row proofs without hiding their data **6**

7 Use established register facts as proof boundaries **8**

8 Extract the shared 99-node free-carrier tail **9**

9 Share scanner proofs at the correct semantic boundary **10**

10 Share residual-system facts before merging compilers **11**

11 Use a common DAG invariant theorem selectively **12**

12 Generalize the existing translation and reflection interfaces **12**

13 Represent elementary row operations once **13**

14 Simplify object-model proof transport without changing strength **14**

15 Improve the current-window API where coherence already exists **15**

16 Finite checking and generated proof design **15**

17 Verification results and limitations **16**

18 A concrete migration plan **17**

A Archive guide and reproduction **18**

1 Scope, baseline, and what should be preserved

The inspected snapshot is the main-branch commit recorded above. The active source root is `Computability/DiophantineConsistency/Lean`. Its physical source counts are:

Category	Files	Physical lines
Library modules and aggregate import file	326	66,857
Checks and audits	220	7,114
Entire active Lean tree	546	73,971

The classification puts files under `checks/`, and files whose names end in `Checks` or `Audit`, in the second row. Physical lines include comments and blank lines. These are source-volume figures, not counts of independent mathematical arguments or kernel proof-term sizes. The supplied `scripts/audit_sources.py` reproduces the census, token counts, hashes, import edges, and approximate declaration spans.

The two largest generated step files account for 6,567 lines: `SplitCircuitSteps59.lean` has 3,445, and `GapCircuitSteps66.lean` has 3,122. They specialize existing generic evaluation theorems. Their size is relevant to emitted code, build granularity, and review ergonomics, but should not be reported as thousands of independently maintained proofs.

After comments and strings are masked, the active source census finds no `sorry`, `axiom`, or `native_decide` tokens. This lexical result is not an audit of imported axioms. Endpoint axiom checks remain necessary.

1.1 Preserve the existing mathematical contract

The current formalization plan distinguishes older checked endpoints from the newer literal polynomial, native decoding, and internal-ZFC composition work. The README and `LEAN_EQUIVALENCE_PLAN.md` explicitly retain open links. A refactoring must preserve these distinctions; shortening a conditional lemma is not completion of the whole consistency equivalence.[1, 2]

In particular, preserve the witness domain, the literal coefficient data, the exact source syntax, the representation witness count, and the dependence on every input coordinate. Preserve raw-input soundness for the proof checker. In object-model arguments, preserve quantification over a model's entire natural-number object rather than silently restricting to externally standard numerals.

1.2 Use several measures of success

Record hand-maintained proof structure, generated source, declaration count, source bytes, elaboration time, peak memory, and endpoint axioms separately. A short line containing 36 rewrite lemmas can be more burdensome than several well-named local facts. A three-line `decide` proof can be expensive to replay. A new general theorem can be an architectural improvement even if its first clients do not amortize the helper's source length.

2 Prioritized proposals

Order	Change	Why this target is concrete
1	Replace private lookup inductions	Seven same-statement replacements, five files, 40 lines removed; core replacements checked.
2	Normalize half-radix digits algebraically	One quotient/remainder identity replaces separate digit cases and exposes the parity argument.
3	Share coefficient-row proof structure	22 envelope lemmas contain 158 identical arithmetic leaf scripts; keep the coefficient data fixed.
4	Respect earlier circuit register lemmas	Two rewrite lists shrink from 19 to 7 and 36 to 9 entries; generator and output change together.
5	Add one assignment-transport law	Removes repeated `Sum`/`Fin` coordinate extensionality in representation constructors.
6	Share sum-of-squares and cost lemmas	Three identical private evaluator lemmas and an already existing parameterized cost argument can be promoted.
7	Share scanner simulation and invariants	Five scanner families repeat iterator facts; the raw-input and decoded-list interfaces must stay distinct.
8	Extract the common 99-node circuit tail	Exact source-data comparison passes under a concrete renaming; one block proof can serve both families.
9	Generalize DAG translation certificates	Reuse existing node-equation uniqueness and reference relations for positive/natural conversions.
10	Reuse object-model proof wrappers	A completeness/context wrapper occurs 26 times across 17 object-theory files.
11	Use smaller views of existing window facts	The coherent `Facts` record already exists; expose derived fields and named construction.
12	Improve generated finite-check granularity	Reduce repeated unfolding and make the trusted checking boundary explicit; benchmark before promotion.

The ordering favors small, independently reviewable changes. The largest architectural idea is number 8; it should follow the low-risk interface cleanup. These are priorities, not estimated completion dates or claimed global savings.

3 Immediate reuse: eliminate seven private lookup proofs

The following files repeat elementary induction on a list or lookup index: `AnnotatedRename.lean`, `AnnotatedBound.lean`, `AnnotatedProofChecker.lean`, `AnnotatedRuleSide.lean`, and `AnnotatedAssumption.lean`. Lean 4.32.0 already provides the required directions:

```
theorem lookup_mem {α : Type u} {xs : List α}
    {n : Nat} {a : α} (h : xs[n]? = some a) : a ∈ xs :=
  List.mem_of_getElem? h

theorem exists_lookup {α : Type u} {xs : List α}
    {a : α} (h : a ∈ xs) : ∃ n, xs[n]? = some a :=
  List.getElem?_of_mem h
```

The supplied `lookup-cleanup.patch` replaces five proofs of the first kind and two of the second. It leaves declarations, names, statements, and call sites in place. The patch applies to the pinned source and removes exactly 40 physical lines. `LookupCleanup.lean` checks the replacement terms against the pinned core library; its axiom output is recorded. This is a direct low-risk patch, although compiling the five modified repository modules and their downstream endpoints is still a separate integration check.[4]

This observation suggests a focused library-reuse pass. Search private inductive lemmas about lists, finite sums, powers, and iteration before writing new abstraction layers. Prefer replacing an established duplicate to creating a parallel local theorem with slightly different notation. Do not generalize every short private fact: a helper should have a clear reusable semantic role.

4 Move renaming out of expression evaluation

`Representation.lean` already provides `Expr.eval_rename`. However, `Rep.reindex`, both sides of `Rep.conjunction`, and both sides of `Rep.disjunction` repeat the same assignment comparison: rewrite expression renaming, apply congruence, prove function equality, and split a sum index.

The useful common law is

$$\mathrm{natAssignment}(v, w) \circ \mathrm{Sum.map}(f, g) = \mathrm{natAssignment}(v \circ f, w \circ g).$$

It gives the semantic rule:

```
theorem eval_rename_natAssignment
    (e : Expr (σ ⊕ α)) (f : σ → τ) (g : α → β)
    (v : τ → Nat) (w : β → Nat) :
    (e.rename (Sum.map f g)).eval (natAssignment v w) =
      e.eval (natAssignment (v ∘ f) (w ∘ g)) := by
  rw [Expr.eval_rename, natAssignment_comp_sumMap]
```

The assignment lemma itself is proved by `funexti;casesi<;>rfl`. The left and right witness layouts, using `Fin.castAdd` and `Fin.natAdd`, become instances. The supplied `AssignmentTransport.lean` contains the general law and both instances.

Orient simplification toward removing a renaming from the expression evaluator and putting it in the assignment. Do not install both directions as simp rules. Keep these rules local or in a small named collection until their effect on other evaluators is clear. Coordinate constructors should expose their semantic interface before the larger arithmetic expression is normalized.

This preserves executable representation data. In particular, replacing a `Rep` constructor with an existentially chosen representation would lose the point of the current design: `Rep` carries a specific expression and finite witness count in `Type`. The desired refactor changes proof fields while preserving those supplied data.

5 Replace half-radix case analysis with one identity

`NativeCyclicRotation.lean` defines a shifted digit by

$$D_{\rho,s}(a,b) = (sa \bmod \rho) + \left\lfloor \frac{sb}{\rho} \right\rfloor,$$

where $a = c(k)$ and $b = c(k-1)$. Its half-radix bounds analyze the possible small digits separately. The arithmetic has a simpler normal form.

Proposition 1. *For natural numbers s, a, b with $s > 0$,*

$$D_{2s,s}(a,b) = s(a \bmod 2) + \lfloor b/2 \rfloor.$$

Consequently, if $b \le 3$, then $D_{2s,s}(a,b) \le s+1$. If also $s \equiv 0 \pmod 2$, then

$$D_{2s,s}(a,b) \bmod 2 = \lfloor b/2 \rfloor.$$

Proof. Multiplication by s commutes with remainder modulo $2s$, and positive multiplicative cancellation gives $\lfloor sb/(2s) \rfloor = \lfloor b/2 \rfloor$. Now $a \bmod 2 \le 1$ and $b \le 3$ imply $\lfloor b/2 \rfloor \le 1$. When s is even, the first summand has even parity and the second is less than 2, giving the final equality. □

The exact Lean proof of the identity is short:

```
theorem step_eq (s a b : Nat) (hs : 0 < s) :
    step s a b = s * (a % 2) + b / 2 := by
  unfold step
  rw [Nat.mul_comm 2 s, Nat.mul_mod_mul_left,
    Nat.mul_div_mul_left _ _ hs]
```

`HalfRadix.lean` checks the identity, bound, and parity statement using only Lean core and its proof-producing `omega` tactic. This is stronger than the existing local bound: the current digit a needs no upper bound; only the predecessor b is constrained. Existing public statements can remain as corollaries, so callers need not change.

The distinction between generality and integration matters. This file proves the scalar arithmetic used by the repository definition. It does not replay the entire cyclic-rotation dependency chain or the later phase argument. Specialization to $s = 2^{88}$ is routine, but should still be compiled in the original module before committing the replacement.

6 Compress the coefficient-row proofs without hiding their data

`NativeCoefficientBounds.lean` contains 22 private `rowEnvelope_0`, …, `rowEnvelope_21` lemmas. Between them, 158 leaves execute the same pattern:

```
rw [abs_le]
constructor <;>
  linarith only [h0, h1, h2, h3, hsq, hmf, hkp]
```

Each row selects a finite piecewise coefficient expression, all bounded by the same envelope. The large source block repeatedly copies the row function, introduces the same three nonnegativity facts, splits its conditional tree, and runs the same linear reasoning. The row data are fixed independently in `NativeQuadraticParent.lean`.

The direct refactoring is one theorem with a row index $k : \mathrm{Fin}(22)$. Case-split k, reduce only the selected coefficient row, decompose the conditional bounds, and close each arithmetic leaf using the shared facts. The pair index stays symbolic; there is no need to enumerate all 528 unordered pairs simply to prove a uniform magnitude bound.

```
-- Schematic proof shape; the candidate is not kernel-verified.
have hsq := sq_nonneg (ports 0)
have hmf := mul_nonneg h3 h0
have hkp := mul_nonneg h1 h0
fin_cases k <;>
  dsimp only [rowCoefficient, Fin.reduceFinMk, Matrix.cons_val] <;>
  repeat' apply abs_ite_le
all_goals
  rw [abs_le]
  constructor <;>
    linarith only [h0, h1, h2, h3, hsq, hmf, hkp]
```

Controlled unfolding is essential. A global `simp` over the parent circuit could unfold much more than these finite coefficient rows. An initial `split_ifs` approach exceeded the simplifier's step limit on this input. The revised proof keeps the existing small `abs_ite_le` lemma and descends through the conditional bound structurally, without adding branch hypotheses. Vector indexing is reduced before this step. The final verification receipt distinguishes checking against exact extracted definitions from a full-module replay.

The supplied whole-file candidate changes 679 physical lines to 130, a 549-line source reduction. This figure is a candidate source measurement; its verification scope is stated separately. Removing private row wrappers is permitted only after confirming they are not used externally. Keep the public `abs_rowCoefficient_le_envelope` and later envelope theorem interfaces.

6.1 Do the same for coefficient naturality

`NativeNinePolynomial52.lean` contains 22 private `mapRowCoefficient` specializations. They establish that a ring homomorphism commutes with each row's coefficient recipe. The mathematical reason is uniform: coefficients use integer literals, addition, subtraction, multiplication, powers, and conditionals whose tests depend on indices rather than ring values.

One finite-row dispatcher with the homomorphism simp laws can replace the duplicated literal functions. A longer-term design stores the coefficient recipe as syntax and derives naturality once by structural induction. Before introducing new syntax, use the existing expression evaluators where suitable. The independent theorem tying those recipes to the actual source rows must remain: redefining the source row to equal its encoded coefficient sum would remove the verification obligation instead of proving it.

The accompanying mapping candidate reduces its full file from 456 to 153 physical lines, a 303-line deletion. The combined isolated coefficient harness was not verified: its last capped run timed out without diagnostics. Both reductions therefore remain candidate source measurements, not completed kernel-checked migrations.

6.2 A stronger abstraction: a finite majorant certificate

The envelope has the nonnegative monomial basis

$$m = (\beta^2,\ M_F\beta,\ M_C,\ M_F,\ \beta,\ K\beta,\ K,\ 1)$$

with weight vector $b = (2, 2, 1, 1, 2, 1, 1, 8)$. Each coefficient branch is an integer linear combination $\sum_i a_i m_i$ with small integer a_i. Prove once that $m_i \geq 0$ and $|a_i| \leq b_i$ imply

$$\left|\sum_i a_i m_i\right| \leq \sum_i |a_i| m_i \leq \sum_i b_i m_i.$$

Then each concrete branch needs only a finite small-integer vector certificate. This can serve multiple coefficient packages and reduces repeated arithmetic reasoning more deeply than a compact tactic script. It is a proposed design, not an implemented result. The basis must retain the existing linear dependence on the large transport port K; an easier bound quadratic in K would not preserve the downstream published scale.

7 Use established register facts as proof boundaries

`SplitCircuitRegisters59.lean` unnecessarily expands some registers whose semantic values have already been proved. The corresponding Gap-family file demonstrates the better discipline. Two particularly clear targets are:

Theorem	Rules before	Rules after	Bytes before	Bytes after
`natural_sourceSRegister`	19	7	744	487
`natural_sourceRankRegister`	36	9	1,142	555

The first should use `natural_sourceHighRegister`. The second should reuse the previously established source-S, source-T, and block-length identities. The generated Lean file shrinks by 844 UTF-8 bytes, from 14,604 to 13,760. Both theorem declarations remain eight physical lines because the rewrite lists were printed on long lines. This is a reduction in exposed dependencies, not a measured physical-line saving.

The supplied patch changes the generator and generated output together. As theorem entries are scheduled, it snapshots the currently available semantic cuts and adds each new cut only for later entries:

```
entries.append((name, idx, expr, dict(cuts), False, defs))
cuts[idx] = name
```

Copying the dictionary is necessary. If an entry retained a reference to the same mutable dictionary, its generated proof could see a later theorem or its own theorem. The improvement depends on respecting source order, not only on finding shorter rewrite lists. Regeneration, Lean checking, and downstream checking are three distinct checks; the archive labels them separately.

8 Extract the shared 99-node free-carrier tail

The most specific larger opportunity is visible in both the mathematical specification and the literal circuit data. The `ratioValue` definitions in `SplitNinePolynomial59.lean` and `GapNinePolynomial66.lean` are byte-identical: 24 lines, 612 bytes. More strongly, the complete natural-circuit suffixes at Split indices $416, \ldots, 514$ and Gap indices $385, \ldots, 483$ are identical after reference renaming. This is 99 nodes: 60 multiplications, 15 subtractions, 17 additions, and 7 literal nodes. There are no input or fixed-coefficient leaf nodes inside the tail.

Internal references map by $j \mapsto j - 31$. The boundary references map as follows:

Split register	Gap register	Role
7	7	Internal input j
9	9	One
10–17	10–17	Shifted external inputs
21	87	Two
409	378	Threshold N
414	383	Rank
415	384	Margin scale

The archive's `inspect_shared_tail.py` compares the pinned generator node dictionaries exactly and emits a JSON receipt. That comparison identifies the target; it is not yet a Lean proof that a new block abstraction preserves evaluation. The script's source inputs and commit are explicit.

8.1 A suitable common interface

Define a small straight-line block whose operand references either a boundary port or an earlier local node. Its 14 boundary ports include the constants one and two; its semantic specification is the common free-carrier finalizer. Then establish four facts:

1. A block evaluator agrees with insertion into a parent topological DAG when the boundary values agree.
2. This one 99-node block evaluates to the common `ratioValue`.
3. Each existing parent suffix is the exact reference-instantiation of the block.
4. The existing front-end register lemmas supply the boundary values.

Insertion should instantiate references directly, adding no copy gates. Retain equality with each pinned parent node list, the operation counts, and the output index. Source-level sharing must not change the measured equation or hide a changed literal coefficient tuple.

The typing issue is real: Split has 12 fixed inputs and Gap has 11, while the existing `NodeRel` is formulated for equal input and fixed-input arities. Since the common tail does not read fixed-input leaves, a boundary-port block captures the exact shared object without coercing the two parent node types.

Put the common block below both specializations in the import graph. It should depend on generic DAG semantics and common finalizer algebra, not on either entire specialized circuit. A module cycle would cancel much of the intended simplification.

This proposal reduces duplicated proof about 99 nodes. It does not remove 99 arithmetic operations from either Diophantine construction, and no such size improvement is claimed. The new generic machinery must be charged against any eventual source reduction.

9 Share scanner proofs at the correct semantic boundary

Five families repeat the same iteration facts: `AnnotatedRenameScan.lean`, `AnnotatedBoundScan.lean`, `AnnotatedLineScan.lean`, `AnnotatedContext.lean`, and `AnnotatedAxiomContext.lean`. Their 15 proofs of `scan_zero`, `scan_succ`, and `scan_done` contain 72 proof-body lines. Mathlib already has the corresponding `Function.iterate` facts. A one-time bridge from the current `Nat.rec` convention can reuse them.[5]

The larger repetition is the simulation of an encoded list by a numeric scanner. The following interface isolates it without replacing the supplied numeric program:

```
structure Encoding (next : A → H → Option H) (step : S → S) where
  state : List A → H → S
  reject : S
  reject_fixed : step reject = reject
  done_fixed : ∀ h, step (state [] h) = state [] h
  cons_step : ∀ row rows h,
    step (state (row :: rows) h) =
      match next row h with
      | none => reject
      | some h' => state rows h'
```

For the associated typed list executor `run`, the common theorem is:

$$|rows| \le f \quad \Longrightarrow \quad \mathrm{iterate}(step, f, state(rows, h)) = \mathrm{result}(\mathrm{run}(rows, h)).$$

The proof is induction on the list. At an empty list, the state is fixed; at a rejected row, rejection is fixed; at a successful row, consume one unit of fuel and apply the induction hypothesis. The prelude-only `ScannerCore.lean` checks this theorem and the accompanying invariant lemmas without additional axioms.

9.1 Do not weaken arbitrary-code soundness

The existing `AnnotatedLineScan.scan_sound` applies to arbitrary natural codes and arbitrary fuel. A theorem about `staterowshistory` covers only states arising from the chosen encoding. The latter cannot replace the former. Keep an invariant on raw numeric states, prove one-step preservation using the actual local checker, and lift it through the common iteration theorem. Use encoded-list simulation for completeness and sufficient-fuel arguments. This separation removes repeated induction without introducing a parser assumption.

The same distinction applies to finished states. The generic interface does not assume that a rejecting state and every completed state are distinct. Any distinction needed by the final acceptance test must remain an explicit property of the concrete numeric implementation.

9.2 Share typed execution and proof fragments

`AnnotatedRename.lean`, `AnnotatedBound.lean`, and `AnnotatedProofChecker.lean` repeat list execution, soundness, and production composition. The checked generic `run_append` and `run_invariant` cover the first layer. A small history-specific layer can cover `Produces.append` and `Produces.step`. Preserve the reversed history convention: executing p and then q builds history in the order $h_q +\!\!+ h_p$ because parent references count backwards. A superficially cleaner forward-list composition would change the checker semantics.

9.3 Factor the supplied PR iterator, not just its existence proof

The bound, line, and axiom-context scanners also repeat program construction from a base, a step, and primitive recursion. A parameterized `iterateProgram` could expose base-history initialization and fuel projection as inputs. This affects executable program data. Preserve or prove the exact `.val` equality, arities, projections, and node-count formulas. Replacing the constructor by an existential proof of primitive recursiveness would discard information required later by the Diophantine representation.

10 Share residual-system facts before merging compilers

Three files contain the same private `eval_sumExpressions` induction: `DAGQuartic.lean`, `NaturalDAGQuartic.lean`, and `QuadraticRegisterSystem.lean`. Publish it in the syntax layer and prove once, for an arbitrary finite residual family r_i, that

$$\left(\sum_i r_i^2\right)(v) = 0 \quad \Longleftrightarrow \quad \forall i,\ r_i(v) = 0 \qquad \text{over } \mathbb{Z}.$$

Each compiler's `expression_zero_iff` then only unfolds its residual family and invokes this interface. The concrete proof is in the proposed `CircuitCoreRefactors.lean`; its checked status is listed separately.

The codomain restriction is substantive. Over a general commutative ring, a sum of squares need not vanish only when each square does. For example, $1^2 + i^2 = 0$ over the complex numbers. The internal-omega theorem in `ObjectResidualSquares` uses the object theory's positive/negative semiring interpretation and order. It cannot be replaced by the integer lemma without an appropriate proof of transfer.

10.1 Promote the parameterized cost bound already present

`QuarticCost.lean` specializes a sum-expression operation bound to a fixed per-expression budget. `DAGQuartic.lean` already proves the parameterized version privately. Promote the latter and retain the old result as a specialization:

$$\big(\forall e \in E,\ \mathrm{binaryOps}(e) \le b\big) \Longrightarrow \mathrm{binaryOps}(\mathrm{sumExpressions}(E)) \le (b+1)|E|.$$

This is reuse of an existing argument rather than new mathematics. Keep the sharper special treatment of the final output residual: applying a coarse uniform bound everywhere can worsen the stated operation bound.

10.2 Keep signed and nonnegative wire realizations distinct

The signed and nonnegative DAG compilers share gate semantics and degree lemmas, but realize wires differently. Signed values use positive and negative natural parts; the single-wire natural compiler requires a nonnegativity invariant. Common RHS syntax and residual lemmas are good targets. A universal compiler interface should carry the required wire realization theorem rather than silently adding nonnegativity to the signed theorem. Exact witness counts remain separate corollaries of the two encodings.

11 Use a common DAG invariant theorem selectively

`ArithmeticDAGSemantics.lean` already proves the local node equation and uniqueness of a value table satisfying those equations. `ArithmeticDAGDegree.lean` and `NaturalDAGQuartic.lean` repeat the corresponding topological strong induction for degree and nonnegativity.

A general invariant has the following mathematical form. Let $P(i, r)$ be a property of register i and value r. If each gate at i preserves P whenever P holds at all its dependency registers, then P holds at every register in the actual run. Topology supplies the strict predecessor bound; `run_node_equation` supplies the gate equation.

```
-- Essential local premise of run_invariant:
∀ i < nodes.length, ∀ values,
  (∀ j ∈ (nodes.getD i (.literal 0)).dependencies,
    P j (values j)) →
  P i ((nodes.getD i (.literal 0)).evalAt p x values)
```

The supplied light `DAGInvariant.lean` and larger `CircuitCoreRefactors.lean` show proposed clients for degree bounds, semantic-table uniqueness, and nonnegative runs. This also supports future range bounds and symbolic normal-form certificates.

This is not an immediate net line-saving result in the measured examples. Three original declarations occupy 42 lines and 2,083 bytes; a proposed helper plus three replacements occupy 45 lines and 2,296 bytes under the supplied formatting. Longer example names contribute to the latter figure. The benefit is concentrating the induction invariant and topology bookkeeping. Adopt it when additional clients justify the common interface, or when it makes a particularly fragile proof substantially easier to maintain.

12 Generalize the existing translation and reflection interfaces

12.1 Positive/natural translation is a simulation, not a ring map

`SplitCircuitTranslation59.lean` and `GapCircuitTranslation66.lean` follow the same proof pattern: handle a short initial prefix, handle one operand-swapped multiplication, then transport all other node equations through a reference map. They use an exceptional register 7 and an input shift. The special swapped nodes occur at Split 452 and Gap 421.

The repository already has `ArithmeticDAG.run_map` and `value_map`. These are valuable for genuine ring homomorphisms, but $x \mapsto x - 1$ is not a ring homomorphism, and these two translations change node lists. Do not try to solve this problem by applying naturality with that shift.

Instead, parameterize the existing simulation proof by its reference map, initial-prefix facts, exceptional-value facts, and node-relation certificate. Extend `NodeRel`, if useful, to allow operand swapping for

addition and multiplication only. Then use existing `run_eq_of_node_equations` or `value_eq_of_node_equations` for the conclusion. Each specialization retains small closed finite certificates and its boundary equations.

The name "natural" describes the external input domain in these files; the node lists still contain negative literals and subtraction. They therefore do not satisfy `NaturalDAGQuartic.Allowed` merely because of their names.

12.2 Do not replace efficient local lemmas by uncontrolled unfolding

The step files contain 999 ordinary step theorems and 193 power theorems. An indexed theorem or a proof-producing local command could generate only needed specializations. However, the current named lemmas provide cached rewrite boundaries. Removing them can force repeated reductions of large literal lists. The meaningful experiment measures cold and incremental replay of actual clients, not only the emitted file size.

`ArithmeticDAGPowers.lean` already provides power certificates, reference relations, and rewrite soundness. `SplitJointPower59.lean` already uses that architecture. Extend those interfaces to blocks and local identities rather than introducing an unrelated reflection framework with a second semantics.

12.3 A small sparse-row normalizer is a better first reflection target

The row-identity families in Gap, Split, and Native each compare low-degree source residuals with coefficient representations. A normalized sparse quadratic syntax is a bounded, reviewable target for a certificate checker. Check its equality to the independently supplied source recipe. Avoid expanding the full high-degree finalizer into a polynomial normal form: that destroys the sharing which the circuit representation was designed to preserve.

12.4 Preserve cancellation-aware degree arguments

Numerical degree labels give upper bounds. Algebraic rewriting can give a strictly better bound by exposing cancellation. The existing `FreeCarrierDegree.lean` and `GapFinalizerDegree66.lean` deliberately exploit such structure. A shorter certificate for a weaker naive degree bound is not a replacement for the stronger theorem. A promising block checker first proves the rewrite identity and then applies the existing weighted-degree interpretation to that chosen normal form. Any Pell or other algebraic hypotheses remain explicit.

13 Represent elementary row operations once

`SplitCertificate59.lean` proves `transformed_zero_iff` by explicitly handling all 22 coordinates in both directions. Most coordinates are unchanged; two rows are modified by adding a different row. The proof is an instance of a very small general fact.

For a row family $r : I \to R$ and distinct indices d, s, define

$$(\mathrm{addAt}_{d,s}\, r)(k) = \begin{cases} r(d) + r(s) & k = d, \\ r(k) & k \neq d. \end{cases}$$

Then

$$(\forall k,\ (\mathrm{addAt}_{d,s}\, r)(k) = 0) \quad \Longleftrightarrow \quad (\forall k,\ r(k) = 0).$$

The forward direction first reads the unchanged source row s, obtaining $r(s) = 0$, and then reads the destination row. No cancellation or inverse operation is necessary: a type with addition and the usual zero laws suffices. Distinctness of d and s is essential to this argument.

`RowOperations.lean` checks this generic theorem and the corresponding fact for a surjective reindexing. The intended concrete refactoring expresses the transformed family as two elementary row additions, then invokes this theorem twice. The final reordering uses the existing `rowOrder_surjective`. The equality between the actual transformed syntax and those elementary operations remains to be established; it is the finite binding certificate, not an assumption to omit.

This is preferable to introducing a full matrix library solely for two row additions. If later transformations include arbitrary invertible linear combinations, a matrix-based zero-set preservation theorem becomes appropriate. The current case supports a much smaller interface.

14 Simplify object-model proof transport without changing strength

The exact expression `completeness_inf_contextZFCax_s[]` appears 26 times across 17 `Object*.lean` files. The repeated proof shell obtains a model, extracts its ZF axioms, establishes validity of the sealed formula, and invokes completeness. Good targets include `ObjectNaturalExpression.lean`, `ObjectCodeClosure.lean`, `ObjectCodeBound.lean`, `ObjectPairingGraph.lean`, and `ObjectResidualSquares.lean`.

Prove a wrapper taking the same model-universe quantification and semantic validity hypothesis and returning the same sealed `BProv` conclusion. Then every caller can focus on its specific arithmetic or coding argument. This theorem should be derived from the current completeness and sealing lemmas; it adds no axiom and gives no license to transfer arbitrary external theorems about standard naturals into ZFC.

14.1 An especially small arity-polymorphism improvement

In `ObjectQuarticCompiler.lean`, `zfc_proves_compiler` proves two versions of the same environment-invariance fact. Its local helper is tied to the input arity m, so the compiled expression's different arity needs a separate `closedTarget` proof. Generalize

```
have closed (f : Expr (Fin m)) : ... := ...
```

to

```
have closed {n : Nat} (f : Expr (Fin n)) : ... := ...
```

and use it at both arities. The body already uses the arity-independent `sat_noZeroForm_iff` and `Sat_sentence_inv`. This proposed edit is source-inspected; the package does not claim a fresh build of its full object-theory dependency chain.

14.2 Build on the algebraic interfaces already present

The project already has `ObjectOmegaSemiring`, `ObjectNaturalOrder`, `GatesSemiring`, and `QuarticSemiring`. The next step is to share model-independent finite-index and renaming facts between their callers.

For example, conjunction of row families has the same index argument in `ObjectQuadraticSystem` and `QuadraticRegisterComposition`.

Two constraints affect the design. First, directly importing `ObjectNaturalOrderedSemiring` into `ObjectResidualSquares` would create a cycle through `ObjectNaturalDiscrete`. Extract lower-level algebraic lemmas with only the needed assumptions or deliberately reorder the modules. Second, `ObjectNaturalExpression.evalRel_unique` currently does not assume every input lies in omega. A replacement proof that requires all inputs to be natural would weaken that theorem. Totality and uniqueness need not use exactly the same abstraction.

15 Improve the current-window API where coherence already exists

`CurrentPolynomialWindow2011.lean` already contains a substantial `Facts` structure tying one source assignment, one window length, one Boolean word, one carry function, and one digit function together. That design should be reused. The proposed improvement is to expose smaller proved views and derived quantities, not to introduce a competing record.

Examples include a named temporal-shift expression, a view containing the four carry/mask facts needed by a local decoding lemma, and named fields for the source-coordinate transport. Construct `Facts` using named fields instead of a long positional tuple when this helps future edits. The latter does not necessarily shorten source, but makes an added field easier to audit.

`CurrentDecodedWindow2011.lean` correctly derives count and phase from the same arithmetic facts before invoking the cell decoder. Keep that order. A "simplified" cell interface that assumes occupation or phase before the relevant polynomial theorem supplies them would make the composition circular. In particular, the decoded-cell alternatives still allow empty cells; the new interface must not quietly assume that every cell selects a tile.

At the arithmetic boundary, derive reusable conversion facts once. Natural subtraction is truncated, integer subtraction is not, and `ZMod` values are representatives modulo the full window length. An unconditional rewrite between these interpretations is usually wrong. Good helper statements carry the exact positivity or range premise that makes the conversion valid. Then use `omega` for the remaining linear natural-number arithmetic and `ring` for equalities after entering the ring domain.

This keeps representation changes separate from mathematical content. It is also more robust than adding long lists of low-level cast and subtraction lemmas to every downstream `simp` call.

One especially local example is congruence weakening. The result `CurrentClause2011.reflected_weight_mod` works modulo 2048; `CurrentEarlyPadding2011.reflected_mod_sixteen` repeats the same selector/nonselector analysis modulo 16. The latter should follow through `Nat.ModEq.of_dvd`, since $16 \mid 2048$. The early-padding module already imports the clause module. The two local selector functions have the same defining equations; their definitional identification or a shared definition must be checked. This is a source-inspected one-line-wrapper opportunity, not a replayed project edit.

16 Finite checking and generated proof design

The desired trust boundary is ordinary Lean proof checking. A generator may produce a finite table, a reference map, or a proposed algebraic certificate; Lean must verify the assertion that table is used to justify. Keep certificate data independent of the conclusion and preserve the current endpoint axiom audits.

Several practical experiments are worthwhile:

1. **Split large closed checks into stable pieces.** Compare a single `decide` over a large nested finite predicate with a small indexed family of checks or a partitioned certificate. This may change resource use, but neither short source nor many small cases guarantees faster replay.
2. **Reduce data selection before symbolic branching.** For row coefficient proofs, choose the finite row and simplify vector indexing before splitting conditionals. Otherwise a tactic may inspect branches belonging to unselected rows.
3. **Keep high-level cuts explicit.** Do not make large register expansion lemmas global simp rules. Use local lists of established semantic facts and prove small ring identities at the resulting boundaries.
4. **Edit the generator with its output.** Check byte-for-byte regeneration and compile the emitted proof. A passing generator test alone does not certify its mathematical output.
5. **Retain compatibility wrappers when useful.** Public theorem names referenced by checks can remain as thin corollaries of a generic proof. Private wrappers can be removed after confirming their scope.

These recommendations do not require replacing kernel reduction with `native_decide`. The current source avoids that tactic. A tactic or certificate generator should emit ordinary checked proof terms if the existing axiom footprint is to be retained. Changes to trust assumptions should be explicit design decisions, not incidental source-shortening techniques.

17 Verification results and limitations

The archive separates three kinds of evidence: kernel checks of new general lemmas, exact source-data checks, and source-inspected proposals. A success in one category is not promoted to another.

Artifact	Verified scope
`ScannerCore.lean` and audit	Seven general scanner/iteration theorems compiled with Lean 4.32.0; all seven have empty axiom lists. Concrete scanner adapters are not yet integrated.
`LookupCleanup.lean`	Both exact polymorphic replacement statements compiled; each uses only `propext`. Seven-edit patch applies to the pinned source; the five full modified modules were not rebuilt.
`HalfRadix.lean`	Identity, bound, and parity theorems compiled. Axiom lists use only `propext` and, for the bound/parity proofs, `Quot.sound`.
`AssignmentTransportCore.lean`	Polymorphic `Sum.elim`/`Sum.map` composition law compiled; its axiom list is `Quot.sound`. Repository-specific clients are separate candidates.
`RowOperations.lean`	Elementary row-addition and surjective-reindexing theorems compiled. The concrete Split certificate binding is still proposed.
`inspect_shared_tail.py`	Pinned JSON inputs and regeneration of both circuit/step families checked; 99-node reference-renamed equality passes. No common-block Lean semantic theorem is claimed.

Artifact	Verified scope
`split-register-cuts.patch`	Generator/output source transformation and unchanged theorem statements checked. Full generated-module replay remains required.
Coefficient-row candidates	Last combined isolated-definition replay timed out after 90 seconds without diagnostics. The 549- and 303-line deletions are unverified candidates. Exact scope is recorded in `evidence/arithmetic/verification.json`.
Other circuit/model prototypes	Source-grounded proposals; their file-specific status is recorded in `evidence/verification.json`. No whole-project refactoring or build is claimed.

17.1 Resource-limited checks are not performance measurements

The pinned compiler and dependency cache were obtained, and several upstream project modules were built. An attempted targeted build later exited with code 137 while building `SelectiveCertificate52`. The environment's shared memory limit was 8 GiB and was under concurrent load. This is a resource-limited replay, not evidence that the original proof is mathematically wrong or that this particular certificate intrinsically requires more than 8 GiB.

A repository-dependent assignment-transport example also aborted under an explicit 1 GiB Lean cap before any theorem diagnostic. An import-only probe for the arithmetic harness exceeded that cap as well. These observations do not establish a relative cost between the proposed and original proofs. No timing speedup or memory reduction is claimed in this report.

17.2 What the delivered evidence does establish

The elementary general lemmas are actual checked Lean code. The source census and fixed-data comparisons are reproducible. The patches and prototypes give specific theorem-level changes that can be reviewed in the original project. The package also identifies where full imported-module replay, concrete adapter proofs, and endpoint audits remain necessary before integration.

18 A concrete migration plan

First change set: library reuse and local algebra. Apply the lookup patch and run the affected checks. Add the half-radix identity and retain the original bound/parity names as corollaries. Add the assignment transport rule at the existing representation layer. Generalize the local arity-polymorphic helper in `ObjectQuarticCompiler`. These changes have small semantic surfaces and can be reviewed separately.

Second change set: repeated finite proofs. Validate the row-envelope replacement against the original imports; preserve the exact coefficient definition and the public theorem statements. Replace the coefficient-mapping family with one naturality proof. Bind the actual Split row transformation to the small row-operation interface. Promote the existing shared sum-of-squares and cost lemmas. Compile downstream public theorems after each step.

Third change set: shared scanner and circuit infrastructure. Integrate the scanner interface first in RenameScan and BoundScan, where the common shape is clearest, then LineScan and AxiomContext.

Keep all-code soundness and typed completeness separate. For the circuit family, apply the semantic-cut generator change and then extract the 99-node tail with exact parent-list binding theorems. Generalize positive/natural translation after the block reference interface has settled.

At each step compare exact signatures, executable program data, witness counts, and endpoint axioms. For generated code, run its reproducibility check. Measure resource use on the same machine under comparable load only after the relevant modules replay successfully. Report net size after counting new helper modules, adapters, and compatibility wrappers.

A Archive guide and reproduction

The principal files are `report.tex` and `report.pdf`. `README.txt` gives the file inventory and runnable commands. `lean/checked/` contains checked small examples; `lean/candidates/` contains proposed repository-dependent integrations. Patches, source-data checkers, and evidence are in `patches/`, `scripts/`, and `evidence/`. Detailed family notes are in `notes/`.

The entire report is one LaTeX source file. Build it with XeLaTeX twice:

```
xelatex -interaction=nonstopmode -halt-on-error report.tex
xelatex -interaction=nonstopmode -halt-on-error report.tex
```

Use the repository's pinned toolchain and manifest for code replay. The exact Lean release is `leanprover/lean4:v4.32.0`; its binary reports commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The supplied driver checks the standalone core files in a temporary output directory and records command results. Repository-dependent candidates require their imported modules to have been built; they are not advertised as independently self-contained projects.

```
python3 scripts/audit_sources.py /path/to/ProveIt /tmp/source-audit
python3 scripts/inspect_shared_tail.py /path/to/ProveIt
python3 scripts/check_core.py --lean /path/to/lean --out /tmp/core-checks
git -C /path/to/ProveIt apply --check /path/to/lookup-cleanup.patch
```

Inline code snippets omit surrounding namespaces and ambient variables when that improves readability. The complete artifact files and their recorded compiler results determine checked status, not the typography of a snippet. The archive manifest records hashes of deliverables. No binary compiler, Mathlib cache, or unrelated repository material is bundled.

References

[1] ProveIt contributors. DiophantineConsistency README. Pinned snapshot `60e9c5d620c6`, 26 October 2026.

[2] ProveIt contributors. Lean equivalence plan. Same snapshot. Describes the exact checked endpoints and outstanding composition obligations.

[3] ProveIt contributors. Active Lean source tree. Specific modules are linked throughout this report; reproducible file hashes, source spans, and imports accompany the archive.

[4] Lean contributors. Core list lemmas, Lean 4.32.0. In particular `List.mem_of_getElem?` and `List.getElem?_of_mem`.

[5] Mathlib contributors. Function iteration lemmas. Exact Mathlib revision from the repository manifest.

[6] ProveIt contributors. Dependency manifest and toolchain file. These pins determine the environment used for the reported checks.